GUÍA DE LABORATORIO 08

SEGURIDAD CON SPRING SECURITY ROLES Y PERMISOS

Autenticación, autorización y control de acceso en una API REST

CURSO	SEMANA	DURACIÓN	MODALIDAD
Desarrollo Web Integrado	**8**	**3 horas**	**Práctica guiada**

CONTEXTO
Semana 7: JPQL y transacciones. Semana 8: protección de los recursos REST mediante autenticación y autorización. Semana 9: se conservará esta base de usuarios/roles y se reemplazará la autenticación HTTP Basic por JWT.

LOGRO DEL LABORATORIO
Al finalizar la práctica, el estudiante protege una API REST con Spring Security, autentica usuarios almacenados en MySQL, cifra contraseñas con BCrypt y aplica reglas de autorización por rol para diferenciar operaciones de consulta y administración.

1. Propósito y alineación con la semana 8

La Semana 8 del sílabo corresponde a “Seguridad con Spring Security. Configuración de roles y permisos”. En esta guía se toma como base la API de productos e inventario construida en las semanas 6 y 7 y se incorpora una capa de seguridad sin introducir todavía JWT, que corresponde a la Semana 9.

1.1. Capacidades que se desarrollan

- Diferenciar autenticación de autorización y explicar por qué ambas son necesarias.
- Comprender el papel de SecurityFilterChain, UserDetailsService, PasswordEncoder y GrantedAuthority.
- Crear usuarios y roles persistentes en MySQL mediante JPA.
- Almacenar contraseñas utilizando BCrypt, nunca en texto plano.
- Configurar acceso por método HTTP y rol usando authorizeHttpRequests.
- Interpretar correctamente respuestas 401 Unauthorized y 403 Forbidden.
- Verificar reglas de seguridad con Postman y pruebas automatizadas con MockMvc.

1.2. Stack de trabajo

Componente	Uso en la práctica
Java 25	Lenguaje de programación.
Spring Boot 4.1.x	Base de la aplicación y auto-configuración.
Spring Security 7.1.x	Autenticación, autorización y filtros de seguridad.
Spring Data JPA / Hibernate	Persistencia de usuarios, roles y datos de negocio.
MySQL	Base de datos relacional.
Maven	Gestión de dependencias y construcción JAR.
VS Code	Entorno de desarrollo.

Componente	Uso en la práctica
Postman	Pruebas de endpoints protegidos.

2. Marco conceptual mínimo

2.1. Autenticación y autorización no son lo mismo

Concepto	Pregunta que responde	Ejemplo
Autenticación	¿Quién eres?	Validar usuario y contraseña.
Autorización	¿Qué puedes hacer?	Permitir DELETE solo a ADMIN.

IDEA CLAVE
Primero Spring Security autentica al usuario. Después evalúa sus autoridades/roles y decide si puede acceder al recurso solicitado.

2.2. ¿Qué ocurre cuando agregamos Spring Security?

Las peticiones HTTP ya no llegan directamente al controlador. Antes pasan por una cadena de filtros de seguridad. Allí se extraen credenciales, se autentica al usuario, se establece el contexto de seguridad y se evalúan las reglas de autorización.

Flujo conceptual

```
Cliente / Postman
    |
    v
Spring Security Filter Chain
    |-- Autenticación -> UserDetailsService -> MySQL
    |-- PasswordEncoder -> compara hash BCrypt
    |-- Autorización -> rol/permiso requerido
    v
@RestController -> @Service -> @Repository -> MySQL
```

2.3. Roles, autoridades y el prefijo ROLE_

Spring Security representa permisos mediante GrantedAuthority. El método hasRole("ADMIN") es un atajo que verifica la autoridad ROLE_ADMIN. Por ello, en esta práctica los roles se almacenarán como ROLE_USER y ROLE_ADMIN para evitar ambigüedades.

2.4. ¿Por qué BCrypt?

Una contraseña no debe guardarse en texto plano ni cifrarse de forma reversible. BCrypt aplica hashing con salt y factor de trabajo, por lo que la aplicación puede validar una contraseña sin recuperar el valor original. En la base de datos se almacena únicamente el hash.

REGLA DE SEGURIDAD
Nunca imprimas contraseñas en logs, no las incluyas en respuestas JSON y no las almacenes en texto plano.

2.5. 401 vs 403

Código	Significado práctico	Caso típico
401 Unauthorized	No existe una autenticación válida.	Sin credenciales o usuario/contraseña

Código	Significado práctico	Caso típico
		incorrectos.
403 Forbidden	El usuario está autenticado, pero no tiene permiso.	Usuario con ROLE_USER intenta ejecutar DELETE reservado a ROLE_ADMIN.

3. Caso práctico de la semana

Continuaremos con la API de productos. Todos los usuarios autenticados podrán consultar. Solo un administrador podrá crear, modificar o eliminar productos. Además se añadirá un endpoint público para comprobar que la aplicación está disponible sin autenticación.

3.1. Política de acceso

Recurso	Método	Acceso requerido
/api/publico/**	GET	Público (sin autenticación)
/api/productos/**	GET	ROLE_USER o ROLE_ADMIN
/api/productos/**	POST	ROLE_ADMIN
/api/productos/**	PUT / PATCH	ROLE_ADMIN
/api/productos/**	DELETE	ROLE_ADMIN
Cualquier otro recurso	Cualquiera	Usuario autenticado

3.2. Estructura que agregaremos

```
src/main/java/com/utp/tienda/
├── config/
│   ├── SecurityConfig.java
│   └── DatosSeguridadIniciales.java
├── model/
│   ├── Producto.java
│   ├── Usuario.java
│   └── Rol.java
├── repository/
│   ├── ProductoRepository.java
│   ├── UsuarioRepository.java
│   └── RolRepository.java
├── security/
│   └── CustomUserDetailsService.java
└── controller/
    ├── ProductoController.java
    └── PublicoController.java
```

4. Paso 1 - Verificar el proyecto de la Semana 7

Antes de incorporar seguridad, ejecuta la aplicación y verifica que la API de productos funcione correctamente. Si el proyecto no compila o la conexión con MySQL falla, resuelve esos problemas antes de continuar.

- MySQL está iniciado y la base de datos utilizada en semanas anteriores existe.
- La aplicación inicia sin errores.
- GET /api/productos devuelve una respuesta válida.
- Las pruebas de semanas anteriores continúan pasando.

BUENA PRÁCTICA
Realiza un commit antes de introducir Spring Security. Así podrás comparar claramente qué cambió y volver atrás si cometes un error.

5. Paso 2 - Agregar Spring Security

Agrega la dependencia de seguridad y el módulo de pruebas de Spring Security. El primer reinicio después de agregar spring-boot-starter-security protegerá automáticamente todos los endpoints; esto es esperado.

pom.xml - dependencias nuevas

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
   <groupId>org.springframework.security</groupId>
   <artifactId>spring-security-test</artifactId>
   <scope>test</scope>
</dependency>
```

Después de guardar el pom.xml, espera a que Maven descargue las dependencias y ejecuta:

```
mvn clean test
```

Si intentas acceder ahora a un endpoint, Spring Security exigirá autenticación. Más adelante reemplazaremos la configuración automática por nuestras reglas.

6. Paso 3 - Crear las entidades Rol y Usuario

Los usuarios y sus roles se almacenarán en MySQL para que la autenticación no dependa de datos codificados dentro del programa.

src/main/java/com/utp/tienda/model/Rol.java

```
package com.utp.tienda.model;

import jakarta.persistence.*;

@Entity
@Table(name = "roles")
public class Rol {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @Column(nullable = false, unique = true, length = 50)
   private String nombre;

   public Rol() {}

   public Rol(String nombre) {
      this.nombre = nombre;
   }

   public Long getId() { return id; }
   public String getNombre() { return nombre; }
   public void setNombre(String nombre) { this.nombre = nombre; }
}
```

src/main/java/com/utp/tienda/model/Usuario.java

```
package com.utp.tienda.model;

import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "usuarios")
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 80)
    private String username;

    @Column(nullable = false, length = 100)
    private String password;

    @Column(nullable = false)
    private boolean activo = true;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
        name = "usuarios_roles",
        joinColumns = @JoinColumn(name = "usuario_id"),
        inverseJoinColumns = @JoinColumn(name = "rol_id")
    )
    private Set<Rol> roles = new HashSet<>();

    public Usuario() {}

    public Usuario(String username, String password, boolean activo) {
        this.username = username;
        this.password = password;
        this.activo = activo;
    }

    public Long getId() { return id; }
    public String getUsername() { return username; }
    public String getPassword() { return password; }
    public boolean isActivo() { return activo; }
    public Set<Rol> getRoles() { return roles; }

    public void setUsername(String username) { this.username = username; }
    public void setPassword(String password) { this.password = password; }
    public void setActivo(boolean activo) { this.activo = activo; }
    public void setRoles(Set<Rol> roles) { this.roles = roles; }
}
```

7. Paso 4 - Crear los repositorios de seguridad

RolRepository.java

```
package com.utp.tienda.repository;

import com.utp.tienda.model.Rol;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;
```

```
public interface RolRepository extends JpaRepository<Rol, Long> {
    Optional<Rol> findByNombre(String nombre);
}
```

UsuarioRepository.java

```
package com.utp.tienda.repository;

import com.utp.tienda.model.Usuario;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    Optional<Usuario> findByUsername(String username);
}
```

8. Paso 5 - Implementar CustomUserDetailsService

Spring Security necesita convertir el registro Usuario de nuestra base de datos en un objeto UserDetails. El servicio siguiente recupera el usuario y transforma cada rol en una autoridad de Spring Security.

src/main/java/com/utp/tienda/security/CustomUserDetailsService.java

```
package com.utp.tienda.security;

import com.utp.tienda.model.Usuario;
import com.utp.tienda.repository.UsuarioRepository;
import org.springframework.security.core.userdetails.*;
import org.springframework.stereotype.Service;

@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UsuarioRepository usuarioRepository;

    public CustomUserDetailsService(UsuarioRepository usuarioRepository) {
        this.usuarioRepository = usuarioRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username)
            throws UsernameNotFoundException {

        Usuario usuario = usuarioRepository.findByUsername(username)
            .orElseThrow(() ->
                new UsernameNotFoundException("Usuario no encontrado"));

        String[] autoridades = usuario.getRoles().stream()
            .map(rol -> rol.getNombre())
            .toArray(String[]::new);

        return User.withUsername(usuario.getUsername())
            .password(usuario.getPassword())
            .authorities(autoridades)
            .disabled(!usuario.isActivo())
            .build();
    }
}
```

OBSERVA

La capa de seguridad depende de una abstracción (UserDetailsService). Spring Security no necesita conocer nuestra entidad

Usuario directamente.

9. Paso 6 - Configurar PasswordEncoder y SecurityFilterChain

Crearemos una configuración explícita. En esta semana utilizaremos HTTP Basic porque permite concentrarnos en autenticación, roles y permisos. En la Semana 9 se reemplazará el mecanismo de transporte de credenciales por JWT.

src/main/java/com/utp/tienda/config/SecurityConfig.java

```
package com.utp.tienda.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http)
            throws Exception {

        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/publico/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/productos/**")
                    .hasAnyRole("USER", "ADMIN")
                .requestMatchers(HttpMethod.POST, "/api/productos/**")
                    .hasRole("ADMIN")
                .requestMatchers(HttpMethod.PUT, "/api/productos/**")
                    .hasRole("ADMIN")
                .requestMatchers(HttpMethod.PATCH, "/api/productos/**")
                    .hasRole("ADMIN")
                .requestMatchers(HttpMethod.DELETE, "/api/productos/**")
                    .hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}
```

9.1. ¿Por qué desactivamos CSRF en este laboratorio?

CSRF protege principalmente aplicaciones web que mantienen autenticación basada en cookies/sesiones del navegador. Nuestra API REST de laboratorio utiliza autenticación HTTP Basic y política STATELESS desde Postman; por eso se desactiva para que POST, PUT, PATCH y DELETE puedan probarse sin token CSRF. Esto no significa que CSRF “siempre deba desactivarse”; la decisión depende del mecanismo de autenticación y del tipo de cliente.

9.2. El orden de las reglas importa

Spring Security evalúa las reglas en el orden declarado. Deben colocarse primero las reglas más específicas y terminar con una regla general como anyRequest().authenticated().

10. Paso 7 - Crear usuarios y roles de prueba con BCrypt

Para que todos los estudiantes puedan reproducir la práctica, inicializaremos dos usuarios únicamente si todavía no existen. Observa que las contraseñas se codifican con PasswordEncoder antes de almacenarse.

src/main/java/com/utp/tienda/config/DatosSeguridadIniciales.java

```
package com.utp.tienda.config;

import com.utp.tienda.model.Rol;
import com.utp.tienda.model.Usuario;
import com.utp.tienda.repository.RolRepository;
import com.utp.tienda.repository.UsuarioRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.password.PasswordEncoder;

import java.util.Set;

@Configuration
public class DatosSeguridadIniciales {

    @Bean
    CommandLineRunner cargarUsuarios(
            RolRepository rolRepository,
            UsuarioRepository usuarioRepository,
            PasswordEncoder passwordEncoder) {

        return args -> {
            Rol rolUser = rolRepository.findByNombre("ROLE_USER")
                .orElseGet(() -> rolRepository.save(new Rol("ROLE_USER")));

            Rol rolAdmin = rolRepository.findByNombre("ROLE_ADMIN")
                .orElseGet(() -> rolRepository.save(new Rol("ROLE_ADMIN")));

            if (usuarioRepository.findByUsername("usuario").isEmpty()) {
                Usuario usuario = new Usuario(
                    "usuario",
                    passwordEncoder.encode("Usuario123*"),
                    true
                );
                usuario.setRoles(Set.of(rolUser));
                usuarioRepository.save(usuario);
            }

            if (usuarioRepository.findByUsername("admin").isEmpty()) {
                Usuario admin = new Usuario(
                    "admin",
                    passwordEncoder.encode("Admin123*"),
                    true
                );
                admin.setRoles(Set.of(rolAdmin, rolUser));
                usuarioRepository.save(admin);
            }
        };
    }
}
```

ENTORNO DE LABORATORIO

Estas credenciales son deliberadamente conocidas para fines didácticos. En un entorno real no deben codificarse credenciales iniciales en el repositorio; utiliza secretos, variables de entorno o un proceso seguro de aprovisionamiento.

11. Paso 8 - Crear un endpoint público

PublicoController.java

```
package com.utp.tienda.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.Map;

@RestController
@RequestMapping("/api/publico")
public class PublicoController {

    @GetMapping("/estado")
    public Map<String, String> estado() {
        return Map.of("estado", "API disponible");
    }
}
```

Este endpoint permite comprobar rápidamente que el servidor está operativo sin necesidad de credenciales.

12. Paso 9 - Ejecutar y verificar las tablas de seguridad

```
mvn spring-boot:run
```

En MySQL Workbench ejecuta:

```
USE tienda_db;
SHOW TABLES;
SELECT id, username, activo, password FROM usuarios;
SELECT * FROM roles;
SELECT * FROM usuarios_roles;
```

La columna password debe contener valores hash similares a $2a$... o $2b$..., nunca las contraseñas originales.

13. Paso 10 - Probar la seguridad con Postman

En Postman selecciona la pestaña Authorization, elige Basic Auth e ingresa el usuario y contraseña de cada escenario.

Prueba	Credenciales	Solicitud	Resultado esperado
1	Ninguna	GET /api/publico/estado	200 OK
2	Ninguna	GET /api/productos	401 Unauthorized
3	usuario / Usuario123*	GET /api/productos	200 OK
4	usuario / Usuario123*	POST /api/productos	403 Forbidden
5	admin / Admin123*	POST /api/productos	201/200 según tu controlador
6	admin / Admin123*	DELETE /api/productos/{id}	204/200 según tu controlador
7	admin / contraseña incorrecta	GET /api/productos	401 Unauthorized

13.1. Ejemplo de POST como ADMIN

```
POST http://localhost:8080/api/productos
Authorization: Basic admin / Admin123*
Content-Type: application/json

{
  "nombre": "Monitor 27 pulgadas",
  "categoria": "Tecnologia",
  "precio": 1250.00,
  "stock": 8
}
```

14. Paso 11 - Añadir pruebas automatizadas de autorización

La seguridad también debe probarse. El objetivo no es solo comprobar que “funciona en Postman”, sino evitar regresiones cuando cambie la configuración.

ProductoControllerSecurityTest.java

```
package com.utp.tienda.controller;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
import org.springframework.security.test.context.support.WithMockUser;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import com.utp.tienda.service.ProductoService;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@WebMvcTest(ProductoController.class)
class ProductoControllerSecurityTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private ProductoService productoService;

    @Test
    void listarSinAutenticacion_debeResponder401() throws Exception {
        mockMvc.perform(get("/api/productos"))
            .andExpect(status().isUnauthorized());
    }

    @Test
    @WithMockUser(roles = "USER")
    void listarComoUser_debeResponder200() throws Exception {
        mockMvc.perform(get("/api/productos"))
            .andExpect(status().isOk());
    }

    @Test
    @WithMockUser(roles = "USER")
    void eliminarComoUser_debeResponder403() throws Exception {
        mockMvc.perform(delete("/api/productos/1"))
            .andExpect(status().isForbidden());
    }
}
```

Nota: si tu slice de prueba necesita importar explícitamente SecurityConfig o mockear componentes adicionales, ajústalo de acuerdo con la estructura real de tu proyecto. Lo importante es conservar los escenarios 401, 200 y 403.

15. Paso 12 - Comprender el flujo completo

```
POST /api/productos
    |
    v
SecurityFilterChain
    |
    +--> ¿credenciales válidas? -- no --> 401
    |
    +--> sí: crea Authentication
    |
    +--> ¿tiene ROLE_ADMIN? ----- no --> 403
    |
    +--> sí
    v
ProductoController
    v
ProductoService
    v
ProductoRepository / MySQL
    v
Respuesta HTTP
```

Este flujo resume el punto central de la semana: la seguridad se ejecuta antes de que la lógica del controlador procese el recurso.

16. Errores frecuentes y cómo diagnosticarlos

Síntoma	Causa probable	Qué revisar
401 en todas las solicitudes	Credenciales inválidas o UserDetailsService no encuentra el usuario.	Usuario en BD, hash, username y logs.
403 al hacer POST/DELETE como usuario normal	El usuario está autenticado pero no posee ROLE_ADMIN.	Es el comportamiento esperado.
403 incluso como admin	Rol almacenado sin el prefijo esperado o regla incorrecta.	ROLE_ADMIN, hasRole("ADMIN") y tabla usuarios_roles.
POST responde 403 por CSRF	CSRF sigue activo.	Configuración csrf para API REST de laboratorio.
Contraseña no coincide	Se guardó texto plano o se codificó dos veces.	PasswordEncoder y datos iniciales.
Bucle o error al serializar Usuario/Rol	Se están exponiendo entidades de seguridad como JSON.	No devuelvas Usuario desde endpoints; usa DTO si fuera necesario.
Pruebas WebMvcTest no cargan	Falta configuración o mocks del slice.	Importar SecurityConfig o ajustar MockitoBean.

17. Actividad de consolidación

Implementa una nueva política para un módulo de inventario con tres niveles de acceso:

1. GET /api/inventario/**: ROLE_USER o ROLE_ADMIN.
2. POST /api/inventario/ajustes: solo ROLE_ADMIN.
3. GET /api/publico/estado: acceso público.
4. Cualquier endpoint no declarado: usuario autenticado.
5. Crea una prueba automatizada que demuestre al menos un 401 y un 403.

18. Reto avanzado - Seguridad a nivel de método

Como ampliación, habilita seguridad de método y protege una operación crítica desde la capa Service. Esto complementa, pero no sustituye, las reglas HTTP.

```
// En una clase de configuración
@EnableMethodSecurity
@Configuration
public class MethodSecurityConfig { }

// En un método de servicio
@PreAuthorize("hasRole('ADMIN')")
public void eliminarProducto(Long id) {
    // ...
}
```

JUICIO DE DISEÑO
Evita dispersar reglas sin criterio. Las reglas de acceso por URL/método HTTP suelen vivir en SecurityFilterChain; las reglas que expresan una condición de negocio específica pueden reforzarse a nivel de método.

19. Evidencias de laboratorio

- Captura de las tablas usuarios, roles y usuarios_roles en MySQL.
- Evidencia de que las contraseñas están almacenadas como hash BCrypt.
- Prueba de GET público sin credenciales.
- Prueba 401 al acceder a /api/productos sin autenticación.
- Prueba 200 al listar productos con ROLE_USER.
- Prueba 403 al intentar modificar datos con ROLE_USER.
- Prueba exitosa de modificación con ROLE_ADMIN.
- Ejecución de mvn test con pruebas de autorización aprobadas.

20. Preguntas de reflexión

1. ¿Cuál es la diferencia entre autenticación y autorización?
2. ¿Por qué hasRole("ADMIN") busca una autoridad ROLE_ADMIN?
3. ¿Por qué una contraseña debe almacenarse con un PasswordEncoder?
4. ¿Qué diferencia existe entre un 401 y un 403?
5. ¿Qué responsabilidad cumple UserDetailsService?
6. ¿Por qué las reglas más específicas deben declararse antes que anyRequest()?
7. ¿Por qué en una API REST stateless se puede tomar una decisión diferente sobre CSRF que en una aplicación web con sesión?
8. ¿Qué parte de esta solución podremos reutilizar cuando incorporemos JWT en la Semana 9?

21. Cierre y conexión con la Semana 9

Hasta esta semana, la API ya cuenta con persistencia, consultas, transacciones y control de acceso por usuarios/roles. HTTP Basic es útil para comprender el mecanismo, pero envía credenciales en cada solicitud y depende obligatoriamente de HTTPS en producción. En la siguiente semana se incorporará autenticación basada en JWT: el usuario se autenticará una vez, recibirá un token y las solicitudes posteriores enviarán ese token en el encabezado Authorization: Bearer <token>.

RESULTADO ALCANZADO
La API ya protege sus recursos antes de ejecutar la lógica de negocio, reconoce usuarios persistentes, valida contraseñas mediante hash y aplica el principio de mínimo privilegio mediante roles.

22. Referencias técnicas

- Spring Security Reference 7.1.1 - Authentication, UserDetailsService, authorization de solicitudes HTTP y roles/authorities.
- Spring Boot Reference 4.1.1 - requisitos del sistema y desarrollo de aplicaciones web.